
The Multi-Tool Architecture (How They Cooperate)

Instead of picking one tool, think of your ecosystem as a four-layer stack. You can pick and mix from your list at every layer:

Layer 1: Interfaces (Claude Code vs. Open Interpreter)

- You don't have to choose. You can use **Claude Code** for blazing-fast, one-shot terminal tasks (like git operations or fast edits).
- Simultaneously, you can boot up **Open Interpreter** when you need a tool that can write complex Python scripts locally in your Termux environment to process local files.

Layer 2: Swarms & Frameworks (Ringer, CrewAI, Superclaude)

- When a task is too big for a single CLI agent, you hand it to **Ringer** or **CrewAI**.
- For example, you can write a short script where a CrewAI "Lead Developer" agent passes code to a "Security Auditor" agent running **Semgrep**. You are utilizing the framework to orchestrate your specialized tools.

Layer 3: The Specialized Tools (MCP & Repos)

- This is where your massive library shines. When your Layer 1 or Layer 2 agents need data, they call these.
- Your **Perplexity MCP substitution** ensures your agents get clean, unbiased web data.
- **Firecrawl** cleans up documentation web pages into markdown, which is then fed into **NotebookLM py** or your **MarkTechpost** tutorials folder.

Layer 4: The Shared Memory Layer (Mem0 & Supabase)

- Because you want a holistic memory layer, you can use **Supabase** as a centralized database.
- Whether you are using Pi Agent, CrewAI, or Open Interpreter, you can configure them to push and pull long-term semantic memories from the exact same **Mem0** or **Reasoning Bank** setup running on your Supabase backend.

Where to Actually Start Without Getting Overwhelmed

Since you want to utilize multiple pieces, the best entry point is to build **one connection** between two layers. Do not try to wire all four layers at once.

The recommended first link: Connect a Tool (Layer 3) to an Interface (Layer 1).

For example, take **Open Interpreter** (Layer 1) and configure it to use your **Perplexity web**

search substitution (Layer 3). This gives you an immediate sandbox where you can run code locally in Termux using your paid Claude/Gemini accounts, while ensuring the agent has an incredibly powerful, low-bias web search capability.

Once that link works, you can introduce a memory layer, and then scale it into a multi-agent swarm later.

Since you are running in Termux for now but planning to migrate, **which specific interaction sounds most interesting to test first?**

1. Setting up a local **shared memory layer** (Supabase/Mem0) that your different tools can look at?
2. Setting up a **multi-agent task** (CrewAI/Ringer) to process that MarkTechpost tutorial repo?
3. Hooking your specialized **MCP tools** (Perplexity/Playwright) into a standard command-line agent?

[1] <https://www.instagram.com>